

Taller: Aplicación Web Full Stack (Frontend + Backend + MySQL)

1. Arquitectura del sistema

Frontend → Backend → Base de datos

2. Frontend (HTML, CSS, JavaScript)

Captura datos del usuario y los envía al backend.

```
fetch("http://localhost:3000/guardar", { ... })
```

3. Backend (Node.js)

Recibe los datos y los guarda en MySQL.

```
app.post("/guardar", (req, res) => { ... })
```

4. Base de datos (MySQL)

```
INSERT INTO contactos (nombre, correo, mensaje) VALUES (?, ?, ?);
```


1. ¿Qué es MySQL?

MySQL es un sistema de gestión de bases de datos que permite:

- Almacenar información
 - Organizar datos en tablas
 - Consultar información cuando se necesite
-

2. Rol de MySQL en el taller

En este proyecto, MySQL se encarga de:

Guardar los datos enviados desde el formulario

3. Estructura de la base de datos

Base de datos

```
CREATE DATABASE contactos_db;  
USE contactos_db;
```

Tabla

```
CREATE TABLE contactos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100),  
    correo VARCHAR(100),  
    mensaje TEXT  
);
```

4. Explicación de la tabla

Campo	Tipo	Función
id	INT	Identificador único
nombre	VARCHAR	Nombre del usuario
correo	VARCHAR	Email
mensaje	TEXT	Contenido del mensaje

5. Cómo se conecta con Node.js

En `server.js`:

```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "root",
  database: "contactos_db"
});
```

Explicación

Parámetro	Significado
host	Servidor local
user	Usuario MySQL
password	Contraseña
database	Base de datos a usar

6. Inserción de datos

Cuando el usuario envía el formulario, Node.js ejecuta:

```
INSERT INTO contactos (nombre, correo, mensaje)
VALUES (?, ?, ?);
```

7. ¿Qué significan los ??

Son **parámetros seguros**:

```
db.query(sql, [nombre, correo, mensaje]);
```

- ✓ Evitan errores
 - ✓ Previenen ataques (inyección SQL)
-

8. Flujo completo con MySQL

```
Formulario
↓
JavaScript (fetch)
↓
```

Node.js recibe datos
↓
MySQL ejecuta INSERT
↓
Datos guardados en tabla

9. Cómo verificar en Workbench

Ejecutar:

```
SELECT * FROM contactos;
```

10. Resultado esperado

Se deben ver registros como:

id	nombre	correo	mensaje
1	Juan	...	Hola
2	Ana	...	Prueba

11. Conceptos clave para estudiantes

Concepto	Explicación
----------	-------------

Base de datos	Contenedor de información
---------------	---------------------------

Tabla	Estructura de datos
-------	---------------------

Registro	Fila de datos
----------	---------------

SQL	Lenguaje de consulta
-----	----------------------

INSERT	Guardar datos
--------	---------------

SELECT	Consultar datos
--------	-----------------

12. Explicación sencilla

“MySQL es el lugar donde se guardan los datos del formulario para que no se pierdan. Node.js actúa como intermediario entre la página web y la base de datos.”

Conclusión

MySQL permite que la aplicación:

- Pase de ser temporal
- A tener **almacenamiento real y persistente**

1. ¿Qué es el backend?

El **backend** es la parte del sistema que:

- Recibe datos del usuario
- Procesa información
- Se conecta con la base de datos

En este taller se utiliza:

- **Node.js** (entorno de ejecución)
 - **Express** (framework para servidor)
-

2. Creación del proyecto backend

Paso 1. Crear carpeta

En el equipo:

```
mkdir backend-contacto  
cd backend-contacto
```

Paso 2. Inicializar proyecto

```
npm init -y
```

Resultado

Se crea:

```
package.json
```

Este archivo gestiona las dependencias del proyecto.

3. Instalación de dependencias

```
npm install express mysql2 cors
```

Explicación

Librería	Función
express	Crear servidor
mysql2	Conectar con MySQL
cors	Permitir comunicación con frontend

4. Estructura del backend

```
backend-contacto/  
├── node_modules/  
├── package.json  
├── package-lock.json  
└── server.js
```

5. Creación del servidor

Archivo: `server.js`

```
const express = require("express");  
const app = express();  
  
// Ruta básica  
app.get("/", (req, res) => {  
  res.send("Servidor funcionando");  
});  
  
// Iniciar servidor  
app.listen(3000, () => {  
  console.log("Servidor en http://localhost:3000");  
});
```

6. Ejecución del servidor

En terminal:

```
node server.js
```

7. Verificación

Abrir navegador:

```
http://localhost:3000
```


Debe mostrar:

Servidor funcionando

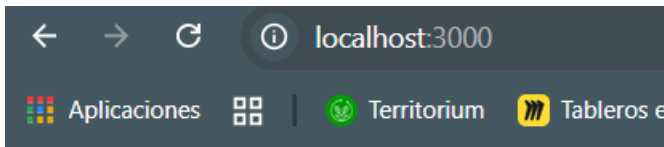
8. Conexión con MySQL

Configuración

```
const mysql = require("mysql2");

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "root",
  database: "contactos_db"
});

db.connect((err) => {
  if (err) {
    console.error(err);
  } else {
    console.log("Conectado a MySQL");
  }
});
```



Servidor conectado a MySQL

9. Middleware necesarios

```
app.use(express.json());
app.use(cors());
```

Función

- `express.json()` → permite recibir datos en formato JSON
 - `cors()` → permite conexión desde el navegador
-

10. Creación de ruta para guardar datos

```
app.post("/guardar", (req, res) => {

  const { nombre, correo, mensaje } = req.body;
```

```
    const sql = "INSERT INTO contactos (nombre, correo, mensaje) VALUES  
(?, ?, ?)";  
  
    db.query(sql, [nombre, correo, mensaje], (err, result) => {  
      if (err) {  
        return res.status(500).send("Error");  
      }  
  
      res.send("Datos guardados correctamente");  
    });  
  });  
});
```

11. Flujo del backend

Recibe petición (POST)
↓
Lee datos (req.body)
↓
Ejecuta SQL
↓
Guarda en MySQL
↓
Responde al frontend

12. Resultado esperado

El backend permite:

- Crear un servidor local
 - Recibir datos del formulario
 - Guardarlos en base de datos
 - Responder al cliente
-

13. Conceptos clave para el estudiante

Concepto	Explicación
Servidor	Programa que responde peticiones
Ruta	URL que ejecuta una acción
POST	Método para enviar datos
req.body	Datos recibidos
SQL	Lenguaje para base de datos

Conexión del Frontend con el Backend usando `fetch`

1. ¿Qué se está haciendo?

El frontend (JavaScript en el navegador) necesita **enviar los datos del formulario al backend**.

Esto se logra mediante:

```
fetch(...)
```

2. Concepto clave

`fetch` permite:

- Enviar peticiones HTTP
 - Comunicarse con el servidor
 - Transferir datos (JSON)
-

3. Flujo de comunicación

Formulario → JavaScript → `fetch` → Backend (Node.js) → MySQL

4. Ubicación en el código

En el archivo:

```
js/script.js
```

Dentro del evento:

```
formulario.addEventListener("submit", ...)
```

5. Código explicado paso a paso

```
fetch("http://localhost:3000/guardar", {
```

- URL del backend
 - Ruta creada en Node.js (`/guardar`)
-

```
method: "POST",
```

- Tipo de petición
- Se usa POST porque se envían datos

```
headers: {  
  "Content-Type": "application/json"  
},
```

- Indica que los datos van en formato JSON

```
body: JSON.stringify({  
  nombre: nombre,  
  correo: correo,  
  mensaje: mensaje  
})
```

- Convierte los datos a texto JSON
- Se envían al backend

6. Recepción de respuesta

```
.then(res => res.text())  
.then(data => {  
  console.log(data);  
})
```

- `res` = respuesta del servidor
- `data` = mensaje enviado por Node.js

7. Manejo de errores

```
.catch(error => {  
  console.error(error);  
})
```

- Captura errores de conexión o servidor

8. Relación con el backend

El backend recibe estos datos mediante:

```
app.post("/guardar", (req, res) => {
  const { nombre, correo, mensaje } = req.body;
});
```

9. Ejemplo completo simplificado

Frontend

```
fetch("http://localhost:3000/guardar", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ nombre, correo, mensaje })
});
```

Backend

```
app.post("/guardar", (req, res) => {
  console.log(req.body);
  res.send("Datos recibidos");
});
```

10. Evidencia para el estudiante

En consola del navegador

Respuesta servidor: Datos guardados correctamente

En consola del backend

Datos recibidos: { nombre: ..., correo: ..., mensaje: ... }

En MySQL

```
SELECT * FROM contactos;
```

11. Conceptos clave

Concepto	Explicación
fetch	Comunicación con servidor
POST	Envío de datos
JSON	Formato de datos

Concepto	Explicación
req.body	Datos recibidos
res.send	Respuesta del servidor